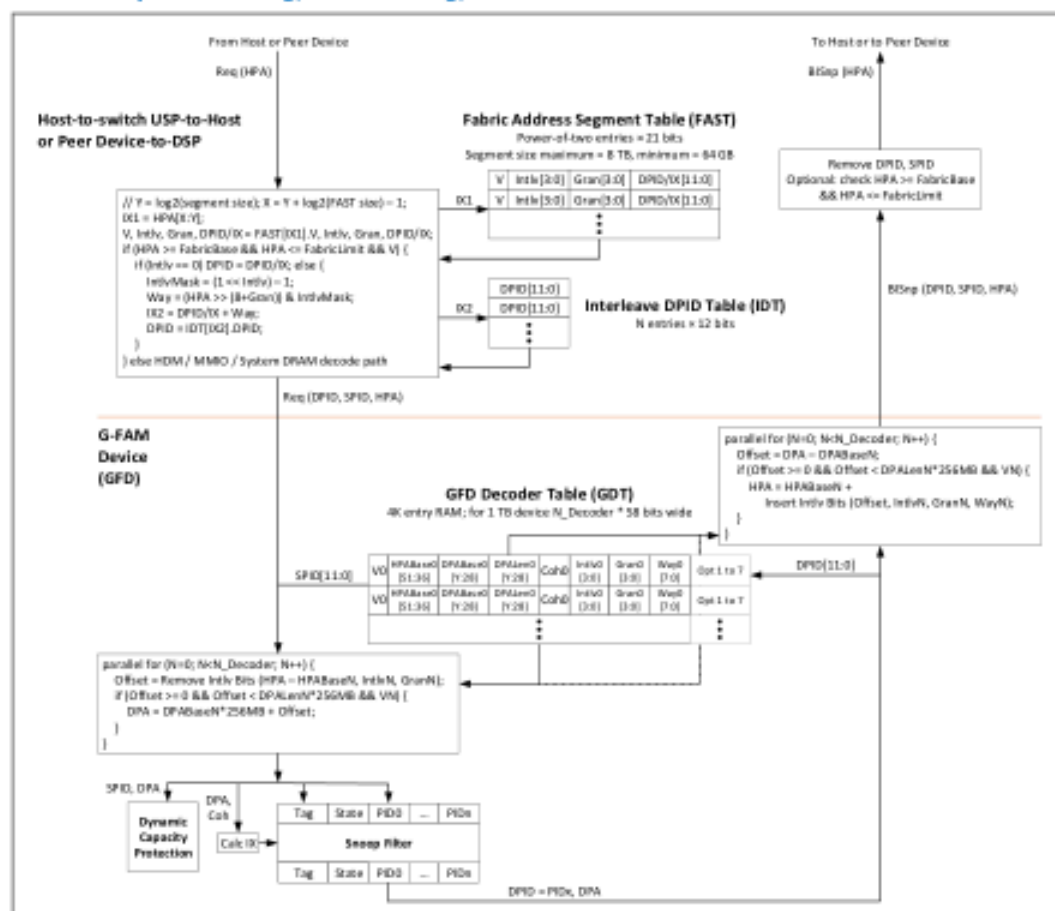


Figure 7-31. G-FAM Request Routing, Interleaving, and Address Translations



The Edge request port shall decode the request HPA to determine the DPID of the target GFD using the FAST¹ and the Interleave DPID Table (IDT). The FAST contains one entry per segment. The FAST depth must be a power-of-two but is implementation dependent. The segment size is specified by the FSegSz[2:0] register as defined in Table 7-81. The FAST entry accessed is determined by bits X:Y of the request address, where Y = log₂ of the segment size in bytes and X = Y + log₂ of the FAST depth in entries. The maximum Fabric Address space and the HPA bits that are used to address the FAST are shown in Table 7-81 for all supported segment sizes for some example FAST depths. For a host with a 52-bit HPA, the maximum Fabric Address space is 4 PB minus one segment each above and below the Fabric Address space for local memory and for MMIO, as shown in Figure 7-29.

1. This section covers using FAST decoders with G-FAM. The LD-FAM Segment Table (LDST) decoders used with LD-FAM have identical functionality with few exceptions. Table 7-81, Table 7-82, and Table 7-83 apply to LD-FAM as well as to G-FAM.

Table 7-81. Fabric Segment Size Table¹

FSegSz[2:0]	Fabric Segment Size	FAST Depth (Entries)			
		256	1K	4K	16K
000b	64 GB	16 TB HPA[43:36]	64 TB HPA[45:36]	256 TB HPA[47:36]	1 PB HPA[49:36]
001b	128 GB	32 TB HPA[44:37]	128 TB HPA[46:37]	512 TB HPA[48:37]	2 PB HPA[50:37]
010b	256 GB	64 TB HPA[45:38]	256 TB HPA[47:38]	1 PB HPA[49:38]	4 PB – 512 GB HPA[51:38]
011b	512 GB	128 TB HPA[46:39]	512 TB HPA[48:39]	2 PB HPA[50:39]	
100b	1 TB	256 TB HPA[47:40]	1 PB HPA[49:40]	4 PB – 2 TB HPA[51:40]	
101b	2 TB	512 TB HPA[48:41]	2 PB HPA[50:41]		
110b	4 TB	1 PB HPA[49:42]	4 PB – 8 TB HPA[51:42]		
111b	8 TB	2 PB HPA[50:43]			

1. LDST Segment Size (LSegSz) uses the same encodings as those defined for FSegSz.

Each FAST entry contains a valid bit (V), the number of interleaving ways (Intlv), the interleave granularity (Gran), and a DPID or IDT index (DPID/IX). The encodings for the Intlv and Gran fields are defined in Table 7-82 and Table 7-83, respectively. If the HPA is between FabricBase and FabricLimit inclusive and the FAST entry valid bit is set, then there is a FAST hit, and the FAST is used to determine the DPID. Otherwise, the target device is determined by other architected decoders.

Table 7-82. Segment Table Intlv[3:0] Field Encoding

Intlv[3:0]	GFD Interleaving Ways
0h	Interleaving is disabled
1h	2-way interleaving
2h	4-way interleaving
3h	8-way interleaving
4h	16-way interleaving
5h	32-way interleaving
6h	64-way interleaving
7h	128-way interleaving
8h	256-way interleaving
9h – Fh	Reserved

Table 7-83. Segment Table Gran[3:0] Field Encoding

Gran [3:0]	GFD Interleave Granularity
0h	256B
1h	512B
2h	1 KB
3h	2 KB
4h	4 KB
5h	8 KB
6h	16 KB
7h – Fh	Reserved

Note that FabricBase and FabricLimit may be used to restrict the amount of the FAST used. For example, for a host with a 52-bit HPA space, if the FAST is accessed using HPA[51:40] without restriction, then it would consume the entire HPA space. In this case, FabricBase and FabricLimit must be set to restrict the Fabric Address space to the desired range of HPA space. This has the effect of reducing the number of entries in the FAST that are being used.

FabricBase and FabricLimit may also be used to allow the FAST to start at an HPA that is not a multiple of the FAST depth. For example, for a host with a 52-bit HPA space, if 2 PB of Fabric Address space is needed to start at an HPA of 1 PB, then a 4K entry FAST with 512 GB segments can be accessed using HPA[50:39] with FabricBase set to 1 PB and FabricLimit set to 3 PB. HPAs 1 PB to 2 PB-1 will then correspond to FAST entries 2048 to 4095, while HPAs 2 PB to 3 PB-1 will wrap around and correspond to FAST entries 0 to 2047. When programming FabricBase, FabricLimit, and segment size, care must be taken to ensure that a wraparound does not occur that would result in aliasing multiple HPAs to the same segment.

On a FAST hit, if the FAST Intlv field is 0h, then GFD interleaving is not being used for this segment and the DPID/IX field contains the GFD's DPID. If the Intlv field is nonzero, then the Interleave Way is selected from the HPA using the Gran and Intlv fields, and then added to the DPID/IX field to generate an index into the IDT. The IDT defines the set of DPIDs for each Interleave Set that is accessible by the Edge request port. For an N-way Interleave Set, the set of DPIDs is determined by N contiguous entries in the IDT, with the first entry pointed to by DPID/IX which may be anywhere in the IDT. The IDT depth is implementation dependent.

After the GFD's DPID is determined, a request that contains the SPID of the Edge request port and the unmodified HPA is sent to the target GFD. The GFD shall then use the SPID to access the GFD Decoder Table (GDT) to select the decoders that are associated with the requester. Note that a host and its associated CXL devices will each have a unique RPID, and therefore each will use a different entry in the GDT. The GDT provides up to 8 decoders per RPID. Each decoder within a GFD Decoder Table entry contains structures defined in [Section 8.2.10.9.10.19](#).

The GFD shall then compare, in parallel, the request HPA against all decoders to determine whether the request hits any decoder's HPA range. To accomplish this, for each decoder, a DPA offset is calculated by first subtracting HPABase from HPA and then removing the interleaving bits. The LSB of the interleaving bits to remove is determined by the interleave granularity and the number of bits to remove is determined by the interleave ways. If offset ≥ 0 , offset $< DPALen$, and the Valid bit is set, then the request hits within that decoder. If only one decoder hits, then the DPA is calculated by adding DPABase to the offset. If zero or multiple decoders hit, then an access error is returned.

After the request HPA is translated to DPA, the RPID and the DPA are used to perform the Dynamic Capacity access check, as described in [Section 7.7.2.5](#), and to access the GFD snoop filter. The design of the snoop filter is beyond the scope of this specification.

When the snoop filter needs to issue a back-invalidate to a host/peer, the DPA is translated to an HPA by performing the HPA-to-DPA steps in reverse. The RPID is used to access the GDT to select the decoders for the requester, which may be the host itself or one of its devices that performs Direct P2P. The GFD shall then compare, in parallel, the DPA against all selected decoders to determine whether the back-invalidate hits any decoder's DPA range.

This is accomplished by first calculating $DPA\ offset = DPA - DPABase$, and then testing whether $offset \geq 0$, $offset < DPALen$, and the decoder is valid. If only one decoder hits, then the HPA is calculated by inserting the interleaving bits into the offset and then adding it to HPABase. When inserting the interleaving bits, the LSB is determined by interleave granularity, the number of bits is determined by the interleaving ways, and the value of the bits is determined by the way within the interleave set. If zero or multiple decoders hit, then an internal snoop filter error has occurred which will be handled as defined in a future specification update.

After the HPA is calculated, a BISnp with the GFD's SPID and HPA is issued to the Edge Port containing the FAST decoder of the host/peer that owns this HDM-DB Region, using the PID stored in the snoop filter as the DPID. The FAST decoder then optionally checks whether the HPA is located within the FAST decoder's Fabric Address space. The DPID and SPID are then removed, and the BISnp is then issued to the host/peer in HBR format.

IMPLEMENTATION NOTE

It is recommended that a PBR switch size structures to support the typical to full scale of a PBR fabric.

It is recommended that the FAST have 4K to 16K entries.

It is recommended that the IDT have 4K to 16K entries to support a sufficient number of interleave groups and interleave ways to cover all GFDs in a system.

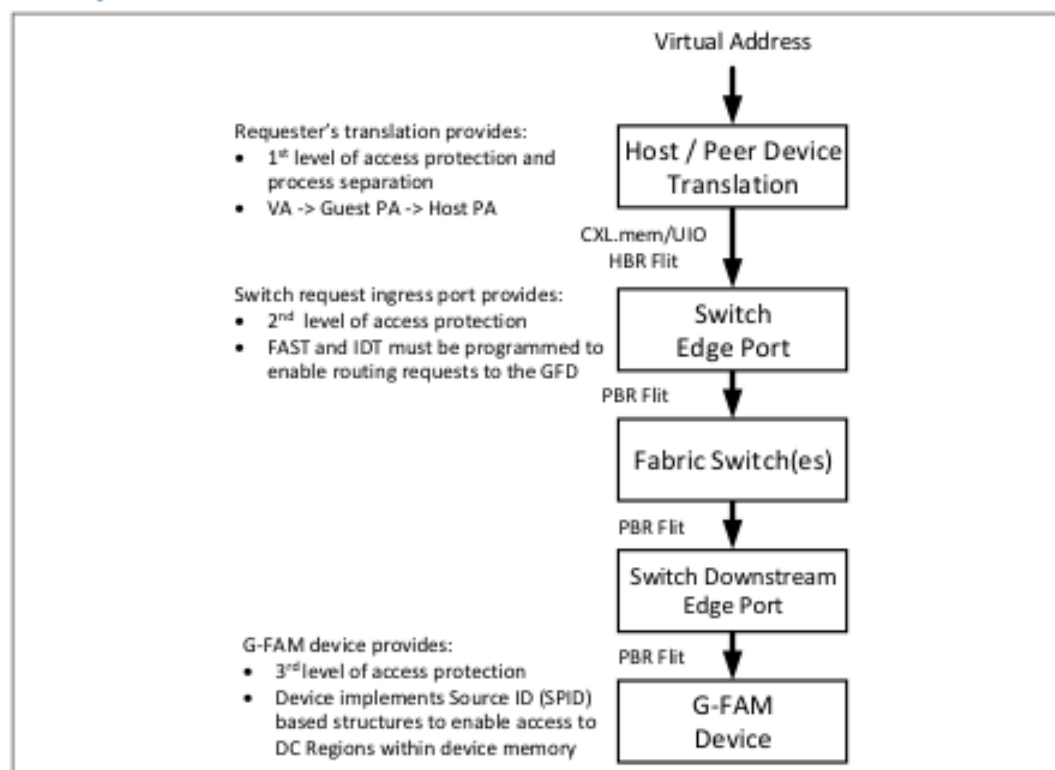
7.7.2.5

G-FAM Access Protection

G-FAM access protection is available at three levels of the hierarchy (see [Figure 7-32](#)):

- The first level of protection is through the host's (or peer device's) page tables. This fine-grained protection is used to restrict the Fabric Address space that is accessible by each process to a subset of that which is accessible by the host/peer.
- The second level of protection is described in the GAE in the form of the Global Memory Mapping Vector (GMV), described in [Section 7.7.2.6](#).
- The third level of protection is at the target GFD itself and is fine grained. This section describes this third level of GFD protection.

Figure 7-32. Memory Access Protection Levels



The GFD's DPA space is divided into one or more Device Media Partitions (DMPs). Each DMP is defined by a base address within DPA space (DMPBase), a length (DMPLength), and a block size (DMPBlockSize). DMPBase and DMPLength must be a multiple of 256 MB, while DMPBlockSize must be a power-of-two size in bytes. The DMPBlockSize values that are supported by a device are device dependent and are defined in the GFD Supported Block Size Mask register. Each GFD decoder targets the DPA range of a DC Region within a single DMP (i.e., must not straddle DMP boundaries). The DC Region's block size is determined by the associated DMP's block size. The number of DMPs is device-implementation dependent. Unique DMPs are typically used for different media types (e.g., DRAM, NVM, etc.) and to provide sufficient DC block sizes to meet customer needs.

The GFD Dynamic Capacity protection mechanism is shown in Figure 7-33. To support scaling to 4096 CXL requesters, the GFD DC protection mechanism uses a concept called Memory Groups. A Memory Group is a set of DMP blocks that can be accessed by the same set of requesters. The maximum number of Memory Groups (NG) that are supported by a GFD is implementation dependent. Each DMP block is assigned a Memory Group ID (GrpID), using a set of Memory Group Tables (MGTs). There is one MGT per DMP. Each MGT has one entry per DMP block within the DMP, with entry 0 in the MGT corresponding to Block 0 within the DMP. The depth of each MGT is implementation dependent. DPA is decoded to determine within which DMP a request falls, and then that DMP's MGT is used to determine the GrpID. The GrpID width is $X = \text{ceiling}(\log_2(NG))$ bits. For example, a device with 33 to 64 groups would require 6-bit GrpIDs.

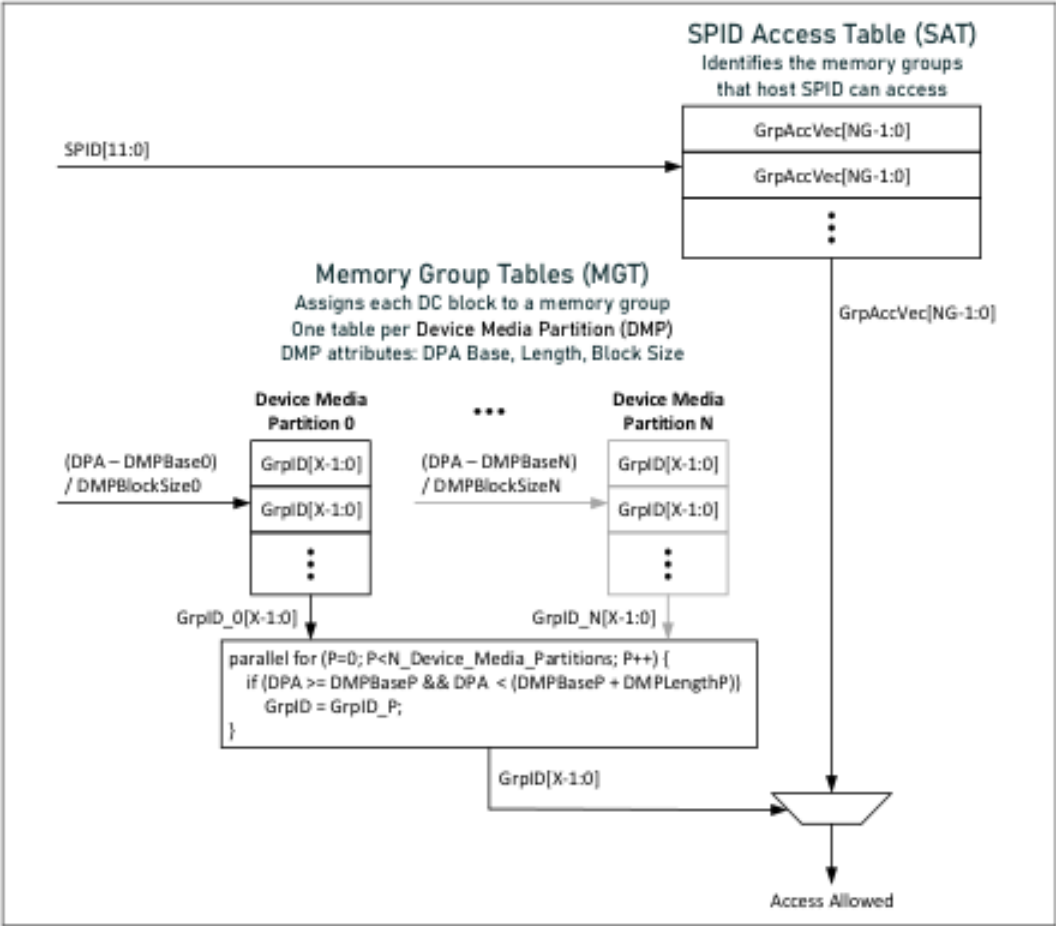
In parallel with determining the GrpID for a request, the Request SPID is used to index the SPID Access Table (SAT) to produce a vector that identifies which Memory Groups the SPID is allowed to access (GrpAccVec). After the GrpID for a request is determined, the GrpID is used to select a GrpAccVec bit to determine whether access is allowed.

IMPLEMENTATION NOTE

To support allocation of GFD capacity to hosts in sufficiently small percentages of the GFD, it is recommended that devices implement a minimum of 1K entries per MGT. Implementations may choose to use a separate RAM per MGT, or may use a single partitioned RAM for all MGTs.

To support a sufficient number of memory ranges with different host access lists, it is recommended that devices implement a minimum of 64 Memory Groups.

Figure 7-33. GFD Dynamic Capacity Access Protections

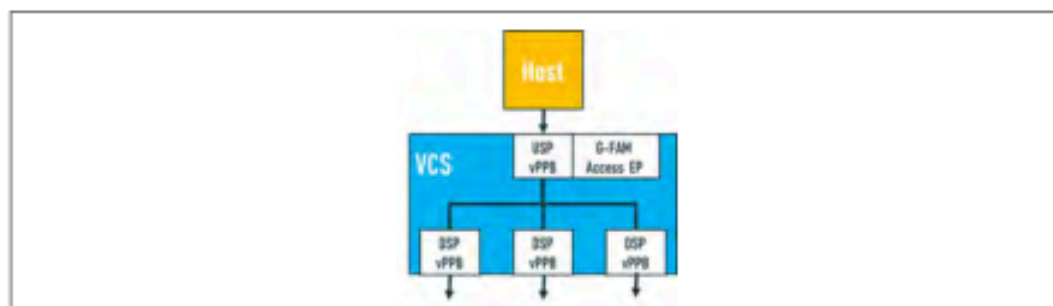


7.7.2.6 Global Memory Access Endpoint

Access to G-FAM/GIM resources and configuration of the FAST through a PBR fabric edge switch is facilitated by a Global Memory Access Endpoint (GAE) which is a Mailbox CCI that includes support for the Global Memory Access Endpoint Command set and the opcodes required to configure and enable FAST use, including **Get PID Access Vectors** and **Configure FAST**. The GAE is presented to the host as a PCIe Endpoint with a Type 0 configuration space as defined in [Section 7.2.9](#).

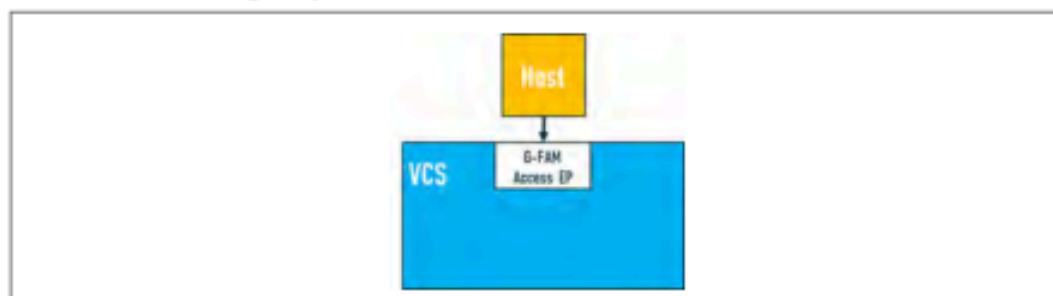
There are two configurations under which a host edge port USP will expose a GAE. The first configuration, illustrated in Figure 7-34, provides LD-FAM and G-FAM/GIM resources to a host. In this configuration, the GAE Mailbox CCI is used to configure G-FAM/GIM access for the USP and any DSPs connected to EPs. It may also include support for opcodes necessary to manage the CXL switch capability providing LD-FAM resources.

Figure 7-34. PBR Fabric Providing LD-FAM and G-FAM Resources



The second configuration, illustrated in Figure 7-35, only provides access to G-FAM/GIM resources. In this configuration, there is no CXL switch instantiated in the VCS and the GAE is the only PCIe function presented to the host.

Figure 7-35. PBR Fabric Providing Only G-FAM Resources



A GAE is also required in the vUSP of a Downstream ES VCS. This GAE is used for configuring that VCS, including configuring the FAST and LDST in the Edge DSPs and providing CDAT information, as described in Section 7.7.12.4.

Each GAE maintains two access vectors, which are used to control whether the host has access to a particular PID:

- **Global Memory Mapping Vector (GMV):** 4k bitmask indicating which PIDs have been enabled for G-FAM or GIM access
- **VendPrefixL0 Target Vector (VTV):** 4k bitmask indicating which PIDs have been enabled for VendPrefixL0

7.7.2.7 Event Notifications from GFDs

GFDs do not maintain individual logs for every requester. Instead, events of interest are reported using the Enhanced Event Notifications defined in Section 8.2.10.2.9 and Section 8.2.10.2.10. These notifications are transported across the fabric using GAM VDMs, as defined in Section 3.1.11.6.

For event notifications sent to a host, the GAM VDM's DPID is the PID of the host's GAE. When received by the GAE, the GAM VDM's 32B payload is written into the host's GAM Buffer. All GAM VDMs that are received by the GAE are logged into the same GAM Buffer, regardless of their SPID.

The GAM Buffer is a circular buffer in host memory that is configured for 32B entries. Its location in host memory is configured with the Set GAM Buffer request. The GAE writes received GAM VDM payloads into the buffer offset that is specified by the head index reported by the Get GAM Buffer request (see [Section 8.2.10.2.11](#)). As the host reads entries, the host increments the tail index using the Set GAM Buffer request (see [Section 8.2.10.2.12](#)). Head and tail indexes wrap to the beginning of the buffer when they increment beyond the buffer size.

The buffer is empty when the head index and tail index are equal. The buffer is full when the head index is immediately before the tail index. Old entries are not overwritten by the GAE until the host removes them from the buffer by incrementing the tail index. The GAE will report a buffer overflow condition if a GAM VDM is received when the buffer is full.

GAM VDMs are not forwarded to peer devices and are instead silently dropped by the peer's edge switch.

7.7.3 Global Integrated Memory (GIM)

A host domain may include multiple tiers of memory:

- Memory natively attached to a host (e.g., DDR, HBM, etc.)
- Device memory attached to a host CXL link
- Device memory attached to a host through CXL switches

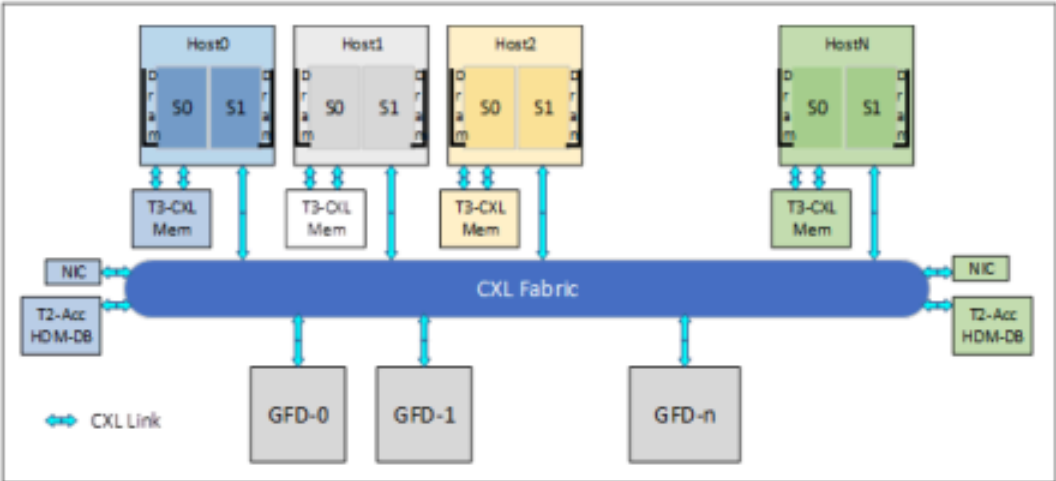
All the memory tiers listed above are managed by a host operating system. CXL devices may be a Type 2 device or Type 3 device and may optionally support back-invalidate channels. A CXL Fabric may be composed of many host domains and G-FAM devices (GFD) as shown in [Figure 7-36](#). GFD is a scalable memory resource that is accessible by all hosts and peer devices within a CXL Fabric.

Each host domain may allow other host domains within the CXL Fabric to access locally managed memory at any tier. Global Integrated Memory (GIM) refers to the memory in remote host domains that is mapped into local host physical address space. Hosts and devices are allowed to initiate cross-domain accesses to GIM, utilizing Unordered I/O (UIO) transactions. CXL.mem or CXL.cache must not be used for GIM accesses.

Cross-domain accesses are considered I/O coherent — data is coherent at the time of access. Remote domains may either mark this memory as uncacheable or manage caches with SW mechanisms.

GIM is primarily used for enabling remote DMA and messaging across domains. It is not intended for memory pooling or borrowing use cases.

Figure 7-36. CXL Fabric Example with Multiple Host Domains and Memory Types

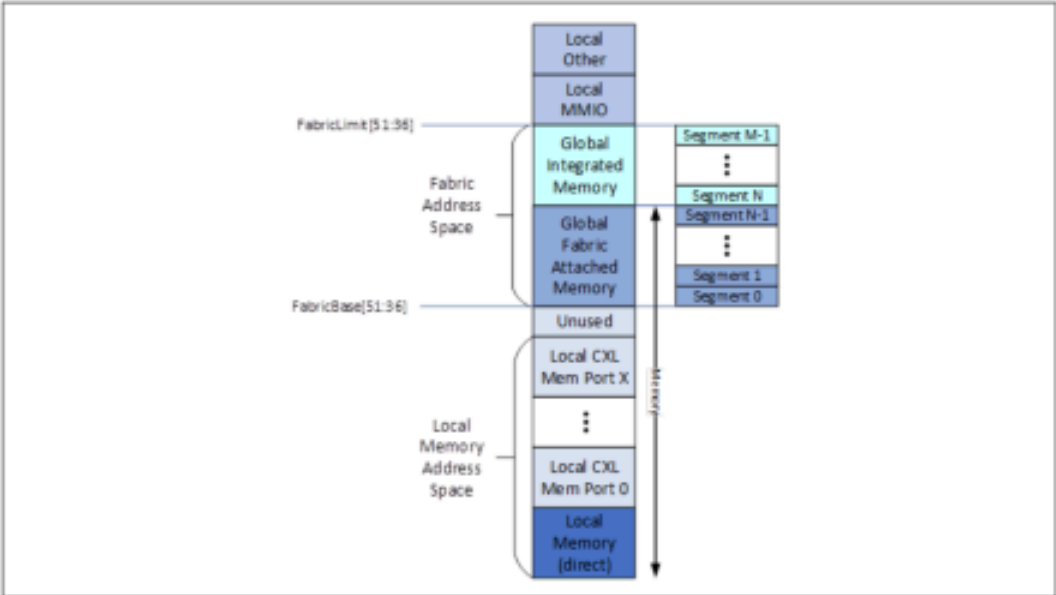


7.7.3.1 Host GIM Physical Address View

Hosts and devices may use proprietary decode mechanisms to identify the target DPID and may bypass address decoders in the switch ingress port. Hosts and devices are typically limited to access between homogeneous peers. See [Section 7.7.3.2](#) for ways by which hosts/devices can access Global Integrated Memory (GIM) without using the FAST decoders. This section covers the decode path that uses the FAST decoders.

Hosts that access GIM and rely on address decoders in the switch must map this range in the Fabric Address Space. Hosts that access GIM and GFD must include both ranges in the Fabric Address Space and must use a contiguous address range within the Host Physical Address (HPA) space as shown in [Figure 7-37](#).

Figure 7-37. Example Host Physical Address View with GFD and GIM



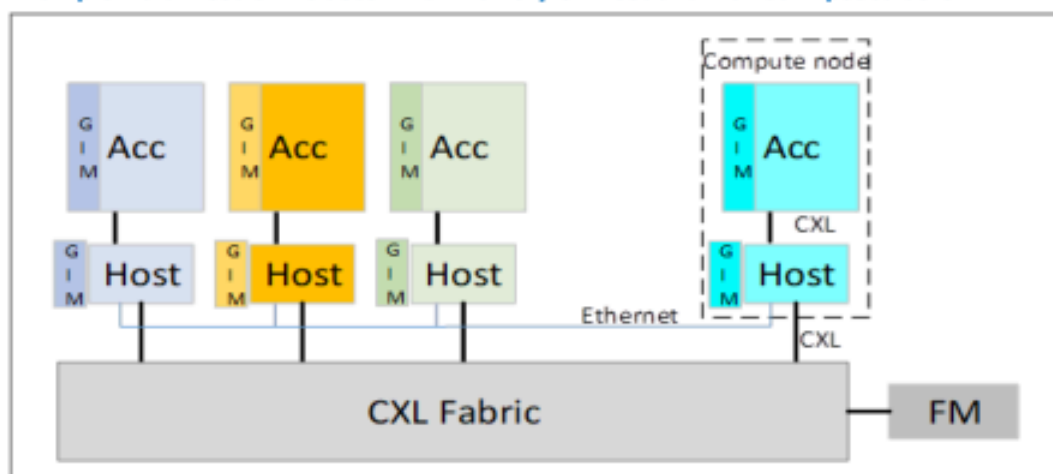
All accesses to GIM regions must only use UIO. It is recommended to map GIM as MMIO instead of a normal write back memory type to avoid potential deadlock. However, implementations may use proprietary methods to guarantee UIO use even when internally using a cacheable memory type. Thus, MMIO mapping of GIM is only a recommendation and not a requirement.

Host and device accesses to GFD and GIM are decoded using a common FAST decoder to determine the target's DPID.

7.7.3.2 Use Cases

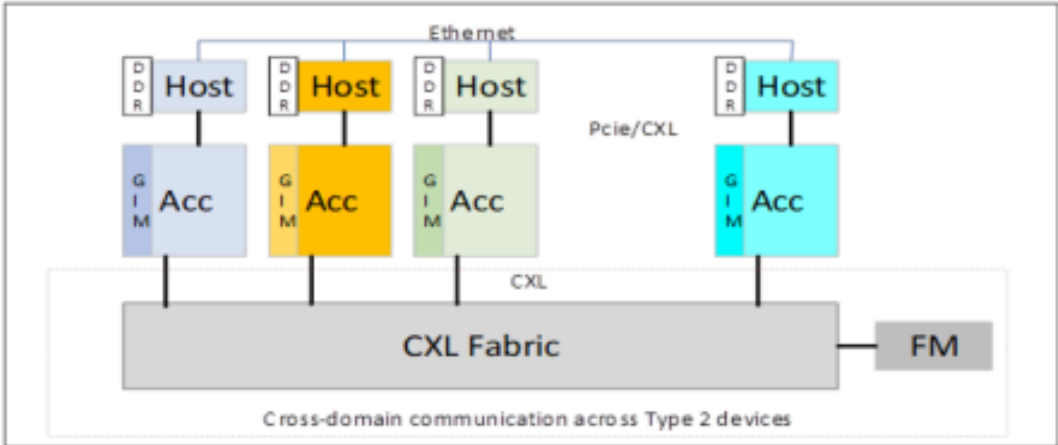
ML and HPC applications are typically distributed across many compute nodes and need a scalable and efficient network for low-latency communication and synchronization. [Figure 7-38](#) is an example of a system with a compute node composed of a Host, an Accelerator, and a cluster of nodes connected through a CXL switch fabric. Each host may expose a region or all available memory to other compute nodes.

Figure 7-38. Example Multi-host CXL Cluster with Memory on Host and Device Exposed as GIM



A second example in [Figure 7-39](#) shows a CXL Fabric that connects all the accelerators. In this example, only the memory attached to the device is exposed to other devices as GIM. UIO allows flexible implementation options to enable RDMA semantics between devices. Software and security requirements are beyond the scope of this specification. GIM builds a framework for using the same set of capabilities for host-to-host communication, device-to-device communication, host-to-device communication, and device-to-host communication.

Figure 7-39. Example ML Cluster Supporting Cross-domain Access through GIM



7.7.3.3 Transaction Flows and Rules for GIM

The flow in Figure 7-40 describes how a host can access GIM in another host, using the fabric address model described earlier in this chapter. While Figure 7-40 uses host-to-host as the example, the same model works for host-to-device, device-to-device and device-to-host as well. A device that implements GIM as target is expected to have the required functionality that translates the combination of <Address: PID> in the incoming UIO TLP to a local memory address and to provide the required security on cross-domain accesses. This functionality can also use more information than just <Address:PID> from the TLP (e.g., PASID) for additional functionality/security. Designs can choose to reuse the GFD architecture for defining this translation/protection functionality or can implement a proprietary IOMMU-like logic. Details of this functionality are beyond the scope of this Specification.

Figure 7-40. GIM Access Flows Using FASTs

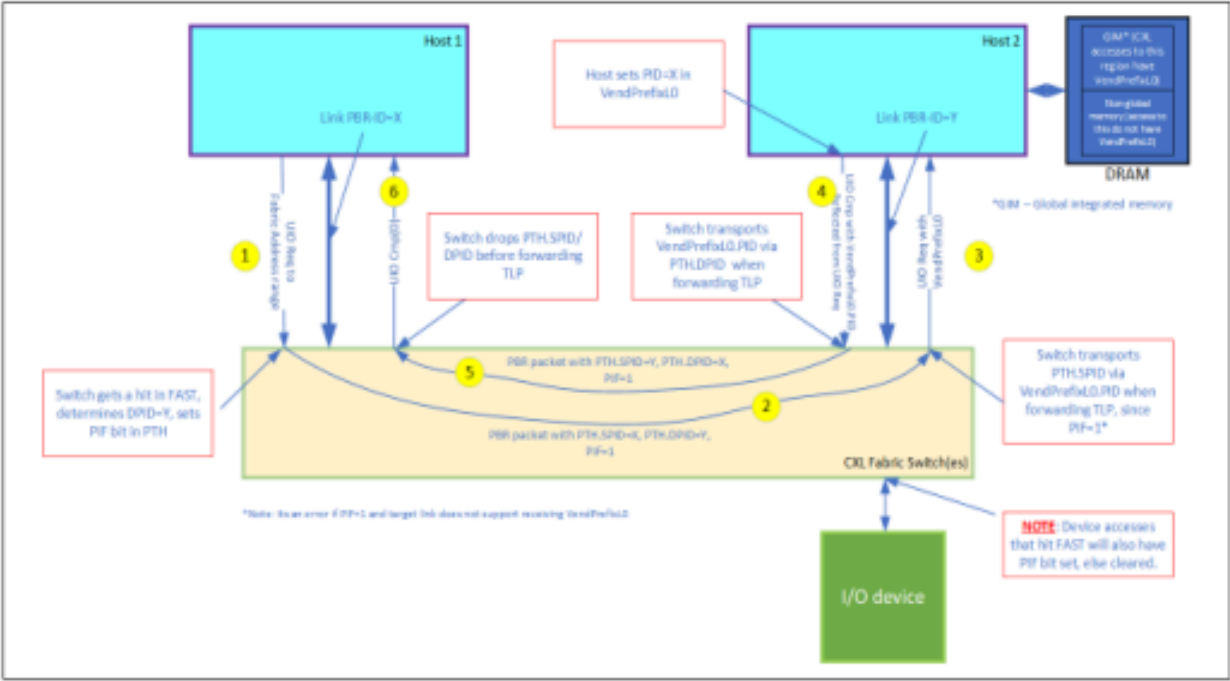
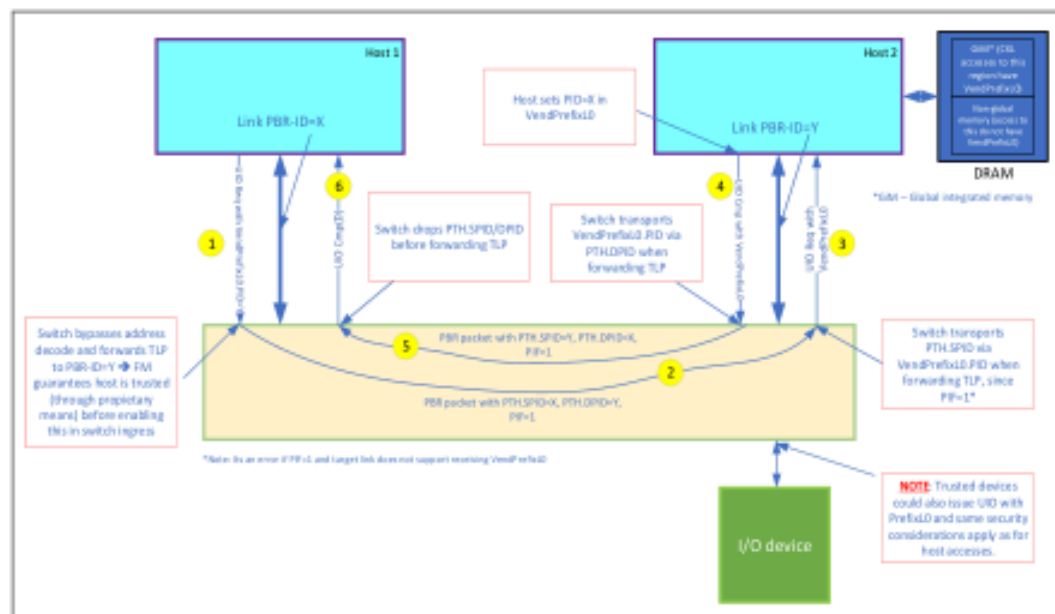


Figure 7-41. GIM Access Flows without FASTs



Although the flows described in [Figure 7-40](#) and [Figure 7-41](#) are self-explanatory, here are the key rules for PBR switches/Hosts/Devices that support the GIM flows:

- FM enables usage of VendPrefixLO on non-PBR edge ports, using the FM API discussed in [Table 7-187](#). By default, VendPrefixLO usage is disabled on edge ports.

The mechanism that the FM uses to determine on which ports to enable this functionality is beyond the scope of this specification.

7.7.3.3.1 GIM Rules for PBR Switch Ingress Port

- GIM flows are supported only via UIO transactions in this version of the specification. At this time, GIM flows are NOT supported via CXL.cachemem transactions or Non-UIO TLPs.
 - If switch ingress port receives a Non-UIO request with VendPrefixL0, it treats it as a UR.
- At the Non-PBR edge ingress port, for UIO request TLPs that do not have VendPrefixL0 and that are decoded via the FASTs, the switch sets the PTH.PIF bit when forwarding the request into the PBR fabric.
 - For UIO request TLPs that are not decoded via the FASTs, this bit is cleared when forwarded to the PBR fabric.
- At the Non-PBR edge ingress port, if the port is enabled for Ingress Request VendPrefixL0 usage and UIO request TLP has VendPrefixL0 and VendPrefixL0.PID matches one of the allowed PIDs in VTV (see [Section 7.7.2.6](#)), the switch bypasses all decode, sets PTH.DPID=VendPrefixL0.PID, PTH.SPID=Ingress Port PID, and PTH.PIF=1 when forwarding the request to the PBR fabric.
 - If a UIO request TLP is received with VendPrefixL0 but the port is not enabled for Ingress Request VendPrefixL0 usage or if the PID in the prefix does not match any of the allowed PIDs in VTV, the switch treats the request as a UR.
- At the Non-PBR edge ingress port, for UIO completion TLPs, the switch forwards the received VendPrefixL0.PID on PTH.DPID when forwarding the packet to the PBR fabric. If Ingress Completion VendPrefixL0 usage is enabled on the port (see

Section 7.7.15.5) and VendPrefixL0.PID matches one of the allowed PIDs in VTV (see Section 7.7.2.6). PTH.SPID on the completion TLP is set to the PID of the ingress port.

- if a UIO completion TLP is received on a Non-PBR edge ingress port when Ingress Completion VendPrefixL0 usage is disabled on the port or if the PID in the prefix does not match any of the allowed PIDs in VTV, the switch must drop the packet and treat it as an Unexpected Completion.
- Switch sets the PIF bit whenever it successfully forwards the received completion TLP to the PBR fabric.

7.7.3.3.2 GIM Rules for PBR Switch Egress Port

- At the Non-PBR edge egress port, for UIO request TLPs with the PTH.PIF bit set, the switch forwards the PTH.SPID field of the request TLP on the VendPrefixL0.PID field if the egress port is enabled for Egress Request VendPrefixL0 usage.
 - If the PTH.PIF bit is set but the egress port is not enabled for Egress Request VendPrefixL0 usage, the switch should treat the request as a UR.
 - If the PTH.PIF bit is cleared in the UIO request TLP, the request TLP is forwarded to the egress link without VendPrefixL0, regardless of whether the port is enabled for Egress Request VendPrefixL0 usage.
- At the Non-PBR edge egress port, the switch does not send VendPrefixL0 on completion TLPs.
- If the Non-PBR edge egress port is in a 'Link Down' state, GIM packets shall be silently dropped.
- Switch forwards the PTH.PIF bit as-is on edge PBR links

7.7.3.3.3 GIM Rules for Host/Devices

- Host/Devices that support VendPrefixL0 semantics and receive a UIO Request TLP with VendPrefixL0 must return the received PID value in the associated completion's VendPrefixL0.
- Host/Devices must always return a value of 0 for Completer ID in the UIO completions.

7.7.3.3.4 Other GIM Rules

- VendPrefixL0 must never be sent on edge PBR links, such as the links connecting to a GFD
- GFD must ignore the PTH.PIF bit on TLPs that the GFD receives
- GFD is permitted to set the PTH.PIF bit on CXL.io request TLPs that the GFD sources and always sets this bit on CXL.io completion TLPs that the GFD sources

Note: If setting the PTH.PIF bit on request TLPs, the GFD must do so only if it is sure that the ultimate destination (e.g., GIM) needs to be aware of the PID of the source agent that is generating the request (such as for functional/security reasons); otherwise, the GFD should not set the bit.

7.7.3.4 Restrictions with Host-to-Host UIO Usages

Host-to-Host UIO usages can result in deadlock when mixed with UIO traffic going to the host that can route back in the host. To avoid such deadlocks:

- Systems that support Host-to-Host UIO must use a separate VC for Host-to-Host UIO traffic vs. remainder of UIO, on host edge links.

(OR)

- Minimally avoid usages that can cause loopback traffic, either in the host or in switches. Generically, this restriction could mean that UIO accesses do not target MMIO space.

A detailed analysis of restrictions that are needed to make a specific system configuration to work with Host-to-Host UIO enabled is beyond the scope of this specification.

A future ECN may be considered that allows for more deadlock avoidance options beyond the two listed above.

7.7.4 Non-GIM Usages with VendPrefixLO

When Hosts/Devices initiate UIO requests with VendPrefixLO, address decoding is bypassed in the Switch ingress port. This allows for proprietary implementations in which the address/data information in the TLP can potentially be vendor-defined. Such usages are beyond the scope of this specification; however, GIM-related rules enumerated in [Section 7.7.3.3](#) allow such implementations as well.

7.7.5 HBR and PBR Switch Configurations

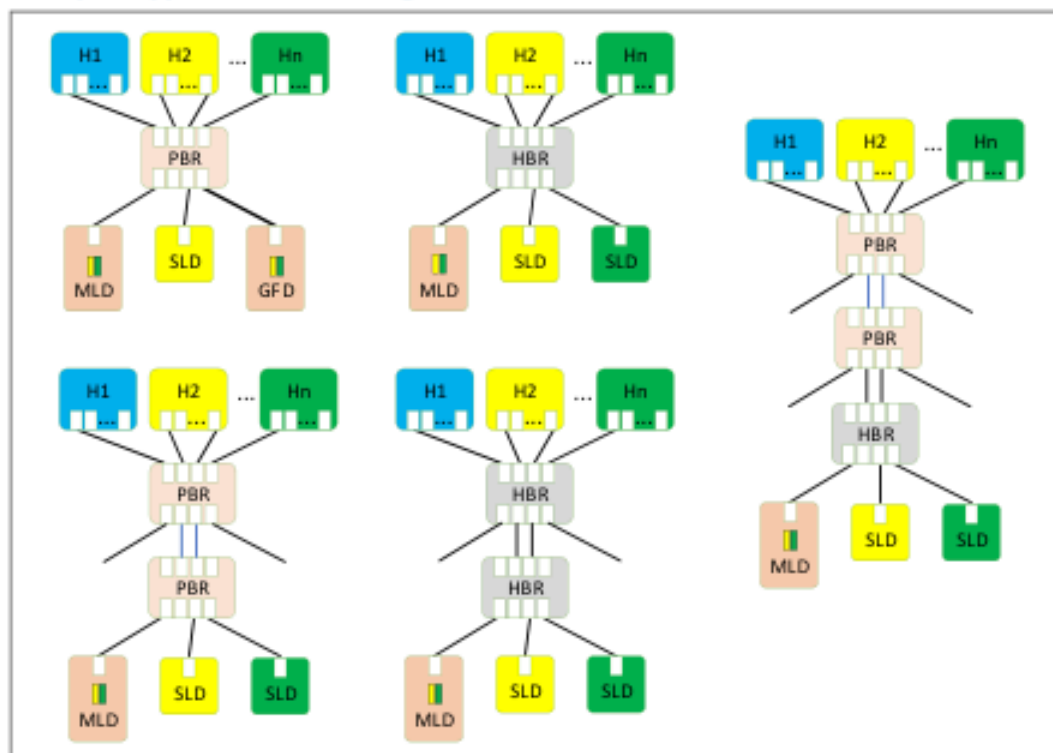
CXL supports two types of switches: HBR (Hierarchy Based Routing) and PBR (Port Based Routing). "HBR" is the shorthand name for the CXL switches introduced in the CXL 2.0 specification and enhanced in subsequent CXL ECNs and specifications. In this section, the interaction between the two will be discussed.

A variety of HBR/PBR switch combinations are supported. The basic rules are as follows:

- Host RP must be connected to an HBR USP, PBR USP, or a non-GFD
- Non-GFD must be connected to an HBR DSP, a PBR DSP, or a Host RP
- PBR USP may be connected only to a host RP; connecting it to an HBR DSP is not supported
- HBR USP may be connected to a host RP, a PBR DSP, or an HBR DSP
- GFD may be connected only to a PBR DSP
- PBR FPort may be connected only to a PBR FPort of a different PBR switch

[Figure 7-42](#) illustrates some example supported switch configurations, but should not be considered a complete list.

Figure 7-42. Example Supported Switch Configurations

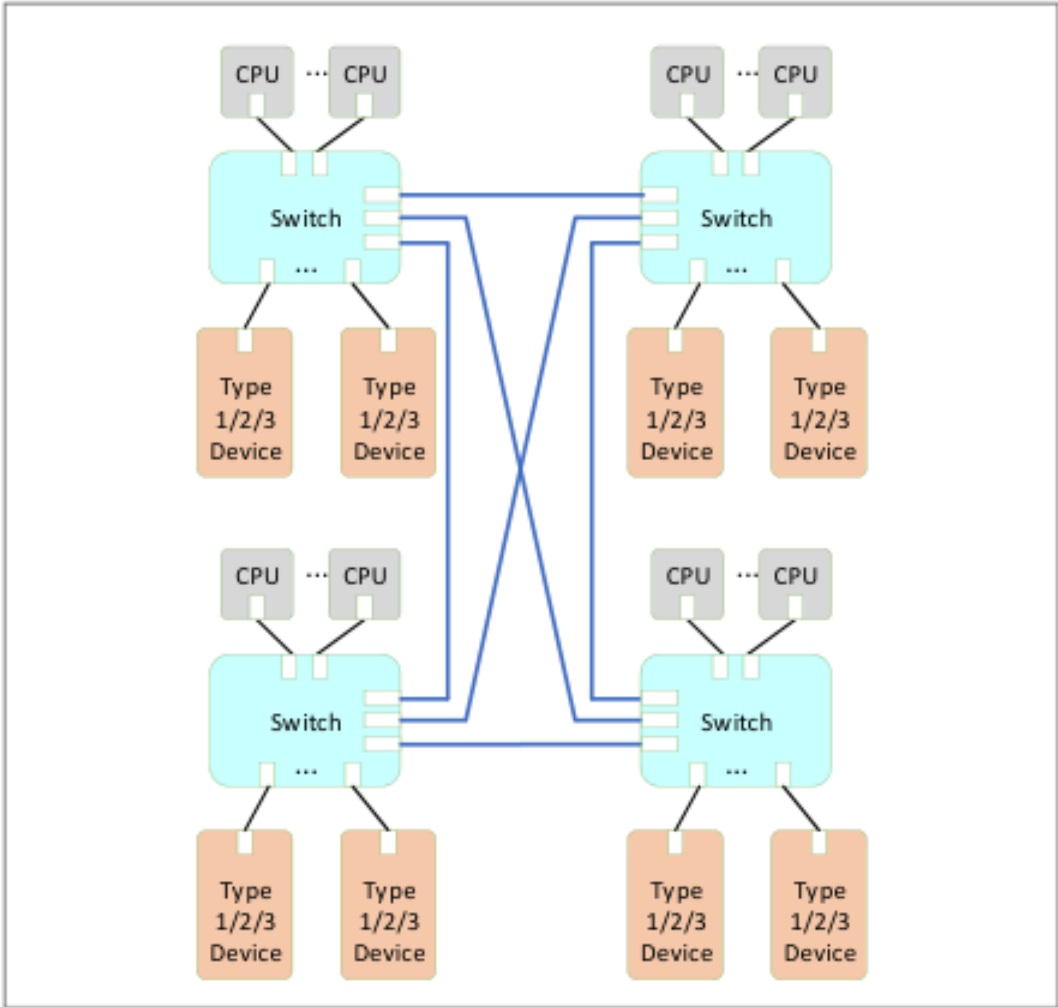


CXL fabric topology is non-prescriptive when using PBR switches. There is no predefined list of supported topologies. PID-based routing combined with flexible routing tables enables a high degree of freedom in choosing a topology. The PBR portion of the fabric may freely use any topology for which deadlock-free routing can be found.

To name a few examples, a PBR fabric might implement a simple PCIe-like tree topology, more-complex tree topologies such as fat tree (aka folded Clos), or non-tree topologies such as mesh, ring, star, linear, butterfly, or HyperX, as well as hybrids and multi-dimensional variants of these topologies.

Figure 7-43 illustrates an example of fully connected mesh topology (aka 1-dimensional HyperX). It has the notable ability to connect a relatively large number of components while still limiting the number of switch traversals. A direct link exists between each pair of switches, so it is possible for the FM to set up routing tables such that all components connected to the same switch can reach one another with a single switch traversal, and all components connected to different switches can reach one another with two switch traversals.

Figure 7-43. Example PBR Mesh Topology



7.7.5.1 PBR Forwarding Dependencies, Loops, and Deadlocks

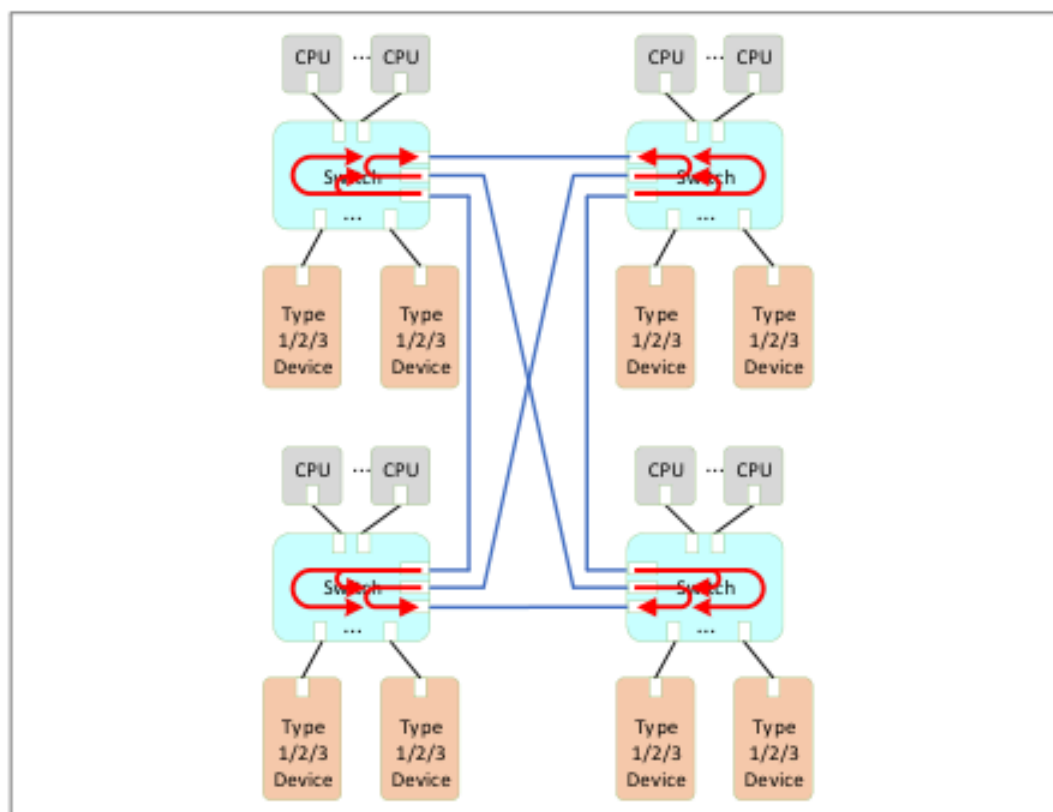
When messages are forwarded through PBR switches from one Fabric Port to another, a dependency is created — acceptance of arriving messages into one PBR Fabric Port is conditional upon the ability to transmit messages out of another PBR Fabric Port. Other arriving traffic commingled on the same inbound link is also affected by the dependency. Thus, traffic waiting to be forwarded can block traffic that needs to exit the PBR portion of the fabric via a USP or DSP of the PBR switch.

Some topologies, such as PCIe tree or fat tree, are inherently free of loops. Thus, the resulting Fabric Port-forwarding dependencies are inherently non-circular. However, in topologies that contain loops, dependencies can form a closed loop, thereby resulting in a deadlock.

The routing table programming in the PBR switches, performed by the FM, must take potential deadlock into account. The dependencies must not be allowed to form a closed loop.

This can be illustrated using the mesh topology presented in [Figure 7-44](#).

Figure 7-44. Example Routing Scheme for a Mesh Topology



One simplistic approach for the mesh topology would be to support only minimal routes. Messages traverse at most one inter-switch PBR link en route from any source host or device to any destination host or device. This simplistic solution is deadlock-free because no message forwarding occurs between PBR Fabric Ports of any switch, and thus there are no forwarding dependencies created from which loops may form. The single route choice, however, limits bandwidth.

[Figure 7-44](#) illustrates a more-sophisticated routing scheme applied to the same mesh topology as [Figure 7-43](#). Each PBR switch is programmed to support three forwarding paths out of the 6 possible pairings. The arrows show permitted forwarding between Fabric Ports. For example, a message traveling from the lower-left switch to the upper-right switch has two route choices:

- Via the direct link
- Indirectly via the upper-left switch

Note that the message cannot travel via the lower-right switch because that switch has no forwarding arrow shown between those Fabric Ports.

The forwarding arrows do not form closed loops; thus, there are no circular dependencies that could lead to deadlock.

This approach to mesh routing (i.e., restricting the choice of intermediate nodes to avoid circular dependencies) can also be applied to larger 1D-HyperX topologies. For a fully connected mesh that contains N switches, there are $N-2$ potential intermediate

switches to consider for possible indirect routes between any pair of switches. However, this deadlock-avoidance restriction limits the usable intermediate switch choices to one-half of that number $((N-2)/2)$, rounding down if N is odd.

Multi-dimensional HyperX topologies can be routed deadlock-free by using this technique within each dimension, and implementing dimension-ordered routing.

Although this section covers some cases for circular dependency avoidance, fully architected deadlock dependency avoidance with topologies that contain fabric loops is beyond the scope of this specification.

7.7.6 PBR Switching Details

7.7.6.1 Virtual Hierarchies Spanning a Fabric

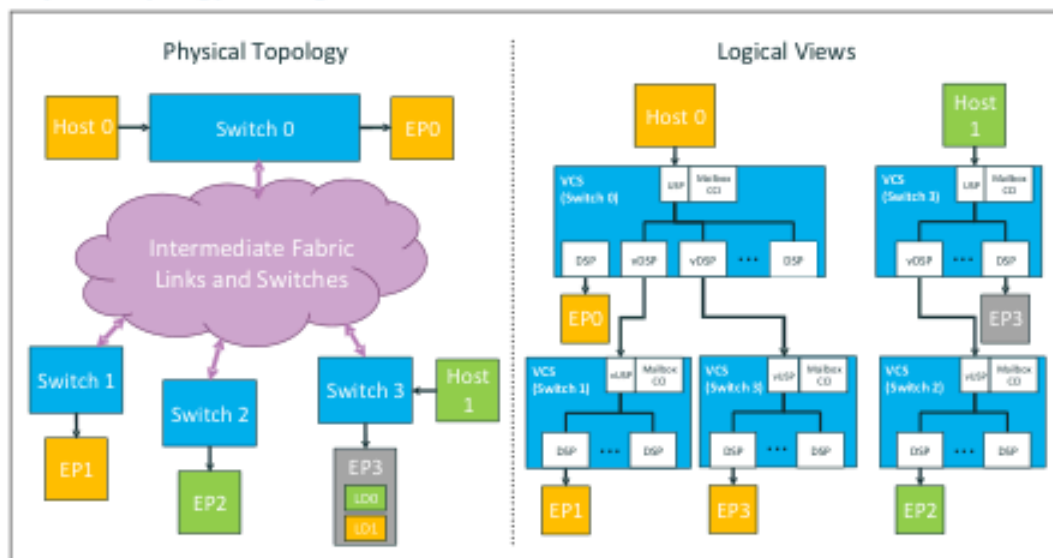
Hosts connected to CXL Fabrics (composed of PBR switches) do not require special, fabric-specific discovery mechanisms. The fabric complexities are abstracted, and the host is presented with a simple switching topology that is compliant with PCIe Base Specification. All intermediate Fabric switches are obscured from host view. At most, two layers of Edge Switches (ESs) are presented:

- **Host ES:** The host discovers a single switch representative of the edge to which it is connected. Any EPs also physically connected to this PBR switch and bound to the host's VH are seen as being directly connected to PPBs within the VCS.
- **Downstream ES:** As desired, the FM may establish binding connections between the Host ES VCS and one or more remote PBR switches within the Fabric. When such a binding connection is established, the remote switch presents a VCS that is connected to one of the Host ES vPPBs. The Host discovers a single link between a virtualized DSP (vDSP) in the Host ES and a virtualized USP (vUSP) in the Downstream ES, regardless of the number of intermediate fabric switches, if any. The link state is virtualized by the Host ES and is representative of the routing path between the two ESs; if any intermediate ISLs go down, the Host ES will report a surprise Link Down error on the corresponding vPPB.
- If an HBR switch is connected to a PBR DSP, that HBR switch and any HBR switches below it will be visible to the host. HBR switches are not Fabric switches.

A PBR switch's operation as a "Host ES" or a "Downstream ES" per the above descriptions is relative to each host's VH. A PBR switch may simultaneously support Host ES Ports and Downstream ES Ports for different VHs. ISLs within the Fabric are capable of carrying bidirectional traffic for more than one VH at the same time. Edge DSPs support PCIe devices, SLDs, MLDs, GFDs, PCIe switches, and CXL HBR switches.

A Mailbox CCI is required in the vUSP of a Downstream ES VCS for management purposes.

Figure 7-45. Physical Topology and Logical View



7.7.6.2 PBR Message Routing across the Fabric

PBR switches can support both static and dynamic routing for each DPID, as determined by message class.

With static routing, messages of a given message class use a single fixed path between source and destination Edge Ports. Messages that use a vDSP/vUSP binding (see [Section 7.7.6.4](#)) always use static routing as well, though the vUSP as a source or destination is always associated with an FPort instead of an Edge Port.

With dynamic routing, messages of a given message class can use different paths between source and destination Edge Ports, dynamically determined by factors such as congestion avoidance, algorithms to distribute traffic across multiple links, or changes with link connectivity. Each DPID supports static routing for those message classes that require it, and it can support either static or dynamic routing for the other message classes.

Dynamic routing is generally preferred when suitable, but in certain cases static routing must be used to ensure in-order delivery of messages as required by ordering rules. Due to its ability to distribute traffic across multiple links, dynamic routing is especially preferred for messages that carry payload data, as indicated in [Table 7-84](#).

Somewhat orthogonal to dynamic vs. static routing, PBR switches support hierarchical and edge-to-edge decoding and routing. With hierarchical routing, a message is decoded and routed within each ES using HBR mechanisms and statically routed between ESs, using vDSP/vUSP bindings. With edge-to-edge routing, a message is routed from a source Edge Port to a destination Edge Port, using a DPID determined at the source Edge Port or GFD. Edge-to-edge routing uses either dynamic or static routing, as determined by the message class.

[Table 7-84](#) summarizes the type of PBR decoding and routing used, by message class.

Table 7-84. PBR Fabric Decoding and Routing, by Message Class

Message Class ** Payload Data		Ordering Rules	Preferred Routing ¹	Decoding and Routing Mechanism
CXL.cache	D2H Req		Dynamic	Edge-to-edge routing using the Cache ID lookups or vPPB bindings
	H2D Rsp	I11a: Snoop (H2D Req) push GO (H2D Rsp)	Static	
	H2D DH **		Dynamic	
	H2D Req	I11a: Snoop (H2D Req) push GO (H2D Rsp)	Static	
	D2H Rsp		Dynamic	
	D2H DH **		Dynamic	
CXL.mem	M2S Req	G8a (HDM-D to Type 2): MemRd*/MemInv* push Mem*Fwd	HDM-H: Dynamic HDM-D: Static HDM-DB: Dynamic	LD-FAM: Edge-to-edge routing if using LDST ² Hierarchical routing if using HDM Decoder G-FAM: edge-to-edge routing using FAST
	M2S Rwd **	-	Dynamic	LD-FAM: Edge-to-edge routing if using LDST ² Hierarchical routing if using HDM Decoder G-FAM: Edge-to-edge routing using FAST
	S2M NDR	E6a: BI-ConflictAck pushes Cmp*	Static	Edge-to-edge routing using vPPB bindings or BI-ID lookups
	S2M DRS **	-	Dynamic	
	S2M BISnp	-	Dynamic	
	M2S BIRsp	-	Dynamic	
	CXL.io	All CXL.io TLPs ** except next row	PCIe (many)	Static
UIO Direct P2P to HDM TLPs **		-	Dynamic	Edge-to-edge routing using FAST or LDST decoder

1. When dynamic routing is preferred, static routing is still permitted.

2. LDST decoders do not support HDM-D.

The Ordering Rules column primarily covers a few special cases with CXL.cachemem messages in which the fabric is required to enforce ordering within a single message class or between two message classes. The alphanumeric identifier refers to ordering summary table entries in [Table 3-57](#) and [Table 3-58](#).

With LD-FAM, host software may use either HDM Decoders or LDST decoders, though LDST decoders do not support HDM-D. Host software implemented solely against the CXL 2.0 Specification comprehends only HDM Decoders, and such host software may continue to use them with PBR Fabrics. Newer host software that comprehends and uses LDST decoders can benefit from edge-to-edge routing, which uses dynamic routing for suitable message classes.

For CXL.io TLPs, the PTH.Hie (hierarchical) bit determines when intermediate PBR switches must use static routing. When the PTH.Hie bit is 1, intermediate PBR switches shall use static routing for the TLP; otherwise, such switches are permitted to use dynamic routing for the TLP. When a PTH is pre-pended to a TLP, the Hie bit shall be 1 if the TLP is a vDSP/vUSP message; otherwise, the Hie bit shall be 0.